

Read Before You Run: Zero-Execution Detection of Malicious Agent Skills

Anonymous Submission

Abstract—Recent large-scale malicious-Skill measurements obtain strong confirmation by executing candidates in sandboxes, synthesizing triggers, and monitoring their behavior. That workflow is valuable for forensics, but too operationally heavy for routine pre-installation decisions by individual developers, registries, and local agents. Static detectors remove the execution burden, yet it remains unclear whether their structured evidence supports precise, source-independent decisions beyond direct semantic review. We study this question under a strict zero-execution constraint. We present *Read Before You Run*, a zero-execution analyzer that separates a Skill’s stated function from its operative instructions and implementation, recovers source-grounded sensitive operations and cross-file data flows, and returns evidence-citing PASS, REVIEW, or BLOCK decisions. The design prioritizes precision: ambiguous evidence is reviewed rather than converted into a blocking decision. On an official 1,000-package source-disjoint benchmark, Ours reaches 98.3% malicious precision and a 0.21% false-positive rate, but its conservative policy sacrifices substantial recall and does not outperform direct document review. Removing the independent high-level function view also does not hurt detection. The deterministic front end scans a separate 1,000-Skill community corpus without failure or target execution. These results establish both the utility and the boundary of lightweight preflight screening: zero-execution analysis can provide low-false-positive, source-grounded triage without sandbox infrastructure, but richer static structure alone does not guarantee stronger source-independent classification.

I. INTRODUCTION

Agent Skills turn natural-language instructions into installable, authority-bearing artifacts. A Skill may bundle persistent directives with scripts, manifests, configuration, and external resources; once admitted, it can influence an agent that already has access to credentials, files, networks, and processes. Installing a Skill is therefore a supply-chain decision. Before asking whether one action is safe, a developer must decide whether the package should enter the agent’s trust boundary at all.

The strongest way to validate an untrusted package is to run it under observation. Recent ecosystem measurements combine static triage with sandbox execution and expert review to confirm malicious Skills at scale [1]. This produces concrete behavioral evidence, but it also requires isolated runtimes, dependency management, trigger generation, instrumentation, and repeated execution. Such infrastructure is appropriate for security laboratories and deep forensic analysis. It is a poor fit for the routine decision faced by an individual developer who is about to install a Skill, a registry screening an upload, or a local agent checking a downloaded package. Moreover,

dormant or environment-specific behavior remains difficult to trigger even inside a sandbox.

This motivates a complementary security layer: *zero-execution preflight screening*. The analyzer should read a Skill as data, require no target runtime or dependency installation, and return a decision before any package-controlled content runs. Deployment simplicity alone, however, is not enough. Skills for development, security, and automation legitimately access files, invoke shells, and contact remote services. A scanner that treats every sensitive primitive as malicious will overwhelm developers with false alarms and make the safer workflow unusable. A practical preflight tool must therefore be conservative about blocking, retain uncertainty, and explain why a package requires review. Figure 1 previews both the threat surface and the resulting community-scale preflight view.

Static Skill detectors have begun to address the execution constraint by recovering instructions, sensitive operations, and cross-file data flows [2]. Richer structure appears attractive: one can first summarize what the Skill claims to do, then ask whether its implementation reads unexpected objects, reaches undeclared destinations, or creates hidden side effects. Yet two simpler explanations can produce the same benchmark gain. A language model may infer the label directly from the primary document, and repository, author, or template cues may correlate with labels without encoding malicious behavior. MaliciousSkillBench shows that performance declines when evaluation separates these provenance cues [3]. Thus, neither a connected static path nor a high benchmark score establishes that structured analysis improves source-independent detection.

We study the following constrained question: *can a zero-execution analyzer provide low-false-positive, source-grounded preflight triage, and does separating stated function from implementation behavior improve detection over direct semantic review under source separation?* The first part captures the operational value needed by individual users: no sandbox and few disruptive false alarms. The second isolates the scientific hypothesis behind the method: whether an engineered function-behavior representation improves the security decision rather than only its explanation.

Read Before You Run answers this question with a bounded static pipeline. It never installs, imports, or executes target content. A parser separates boundary descriptions from operative instructions, code, and configuration. The detector normalizes sensitive objects and operations, constructs cross-file Python call graphs and interprocedural taint summaries, and preserves

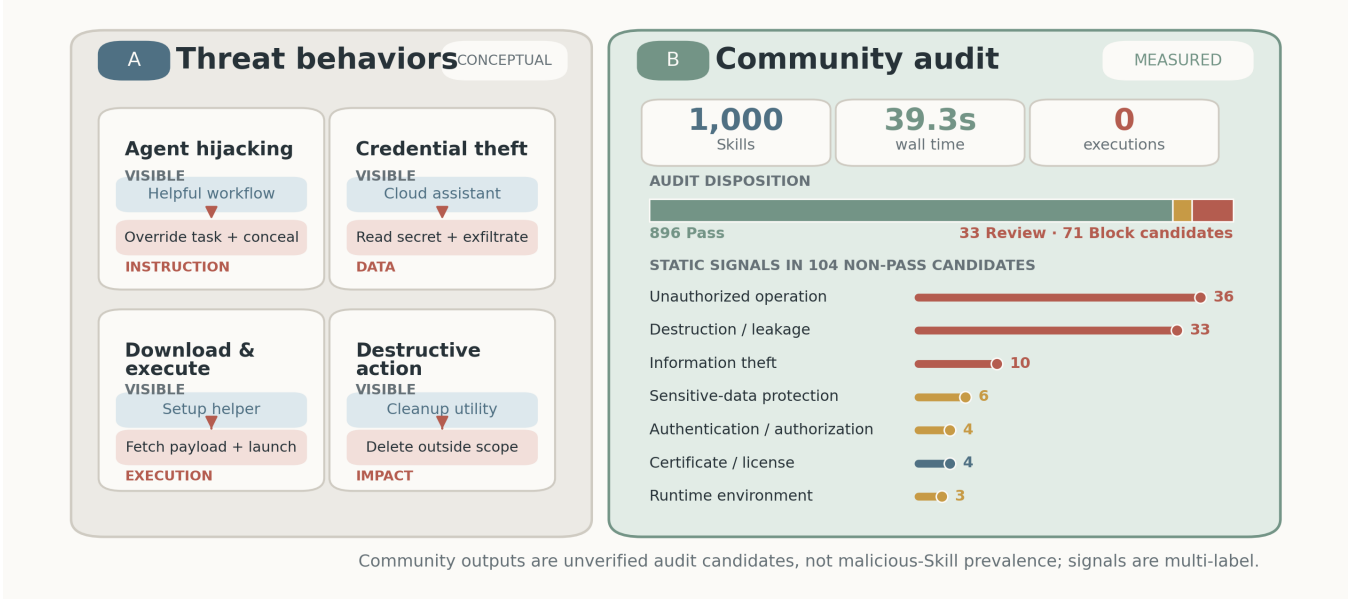


Fig. 1. Threat behaviors and community-scale zero-execution screening. (A) Four representative ways in which a benign-looking Skill can introduce instruction hijacking, information theft, unauthorized execution, or destructive effects. These examples are conceptual. (B) The deterministic front end scans 1,000 current community Skills without target execution and surfaces source-grounded audit candidates. The community corpus is unlabeled; neither the dispositions nor the multi-label signals estimate malicious prevalence.

unresolved calls, unsupported languages, truncation, and invalid model output. A schema-constrained reviewer combines the evidence under a fixed risk taxonomy and returns PASS, REVIEW, or BLOCK. Crucially, uncertainty is not silently mapped to benign, while blocking remains reserved for evidence strong enough to justify disrupting a legitimate Skill.

We evaluate the design on three complementary corpora. A 200-package public benchmark supports controlled diagnosis but is perfectly confounded with source and owner. An official 1,000-package source-disjoint benchmark tests the complete method against frozen rules and a one-call document reviewer. A separate 1,000-Skill community snapshot measures whether the deterministic front end can process current packages without runtime preparation; it scans the full snapshot in 39.3 seconds. The results identify a clear operating boundary. Ours maintains very high malicious precision and the lowest false-positive rate, but transfers substantial ambiguity to review and misses more attacks than direct document classification. The apparent gain on the smaller benchmark disappears under source separation, and removing the independent function view does not reduce detection quality.

These findings do not erase the value of zero-execution analysis; they locate it more precisely. For individual developers, the static front end provides a lightweight way to surface source-grounded risk candidates before installation, without constructing a sandbox or running untrusted code. For researchers, the source-disjoint result separates *evidence production* from *maliciousness prediction*: richer static structure can improve what a reviewer sees without necessarily improving which packages a classifier finds. Future Skill defenses should therefore report false positives, abstention, coverage,

and provenance robustness alongside aggregate accuracy.

This paper makes three contributions:

- A **zero-execution preflight pipeline** that requires no target runtime, separates stated function from implementation behavior, and returns source-grounded findings with explicit unknowns.
- A **precision-oriented admission workflow** that distinguishes BLOCK from REVIEW, supports fast deterministic ecosystem scanning, and avoids treating every sensitive operation as malicious.
- A **provenance-aware evaluation and empirical boundary** across diagnostic, source-disjoint, and community corpora, showing that the method’s low-false-positive operating point is useful for triage even though its structured decomposition does not improve source-disjoint classification.

Section II defines the threat model, Section III reviews adjacent defenses, Section IV presents the detector, and Section V evaluates its supported and rejected claims.

II. BACKGROUND AND THREAT MODEL

A. Agent Skill Packages

An Agent Skill is treated as a versioned package containing a primary skill document and any referenced instructions, scripts, configuration, manifests, and resources. The package boundary is the unit of analysis. A single document may mix user-facing descriptions with operative instructions, so content roles are assigned at section or paragraph granularity rather than by filename alone.

B. Threat Model

The adversary controls every untrusted byte in the package and may distribute behavior across artifacts. The adversary knows the analysis strategy and may use paraphrase, aliases, indirection, string construction, dynamic imports, reflection, simple encoding, or opaque binaries. The detector and its policy and rule versions are trusted. The detector neither installs dependencies nor imports, invokes, or executes target-supplied content, and it does not contact endpoints named by the package.

The method does not claim to recover code obtained only at runtime, behavior inside unavailable dependencies, or semantics hidden in unrecoverable encrypted or binary content. Such uncertainty is represented in the output. In particular, PASS is not a proof of safety.

C. Design Goals

The detector should (G1) remain zero execution; (G2) preserve independence between high-level function and internal behavior; (G3) connect cross-file evidence; (G4) reduce false positives on legitimate sensitive operations; (G5) expose uncertainty instead of silently treating missing analysis as benign; and (G6) return evidence that a reviewer can locate in the original package.

III. RELATED WORK

A. Agent Skills as a Supply-Chain Boundary

Agent Skills occupy an unusual point between prompts and packages: they combine natural-language control with executable code, configuration, and resources. Liu et al. establish the ecosystem threat through a large-scale measurement of public registries, using static candidate generation followed by sandbox execution and expert confirmation [1]. MalSkills instead targets zero-execution detection by extracting security-sensitive operations, linking values across artifacts, and applying neuro-symbolic rules to a dependency graph [2]. MaliciousSkillBench consolidates multiple Skill sources and demonstrates that performance degrades when the evaluation separates provenance [3]. Together, these works define the central tension for admission control: behavioral confirmation requires execution, whereas artifact-only detection must reason under ambiguity and distribution shift.

The broader software-supply-chain literature exposes a complementary entry point. Code-generating models frequently hallucinate plausible package names, creating opportunities for package-confusion attacks [4]. That work asks which dependency an LLM causes a developer to select; malicious Skill detection asks what an already selected package can instruct and enable an agent to do. Our work is closest to MalSkills, but evaluates whether adding separated high-level context to recovered behavior improves detection under a source-disjoint protocol. The result is negative, delimiting this design point rather than treating richer structure as an assumed benefit.

B. Adversarial Instructions, Tools, and Agent State

Prompt injection studies an attacker who places competing instructions in content consumed by an application. Liu et al. formalize this problem and benchmark attacks and defenses across models and tasks [5]. InjecAgent specializes the threat to tool-integrated agents, covering harmful actions and private-data exfiltration triggered by tool-returned content [6]. These attacks occur during a task trajectory, whereas a malicious Skill can make adversarial instructions persistent and package them with helper code or installation behavior.

Other work compromises different components of the agent stack. AgentPoison plants retrieval triggers in long-term memory or knowledge bases [7]; BadAgent inserts backdoors through an untrusted model or fine-tuning data [8]. ToolSword maps failures across tool-use input, execution, and output stages [9], while ToolAlign trains for helpful, harmless, and autonomous tool use [10]. These studies show that agent behavior can be subverted through model weights, memory, retrieved content, or tool feedback. Our adversary instead controls a distributable Skill artifact. The detector therefore cannot retrain the underlying model or observe a trigger at runtime; it must identify suspicious instructions and implementation evidence before the artifact becomes agent state.

C. Dynamic Analysis and Agent Security Evaluation

Dynamic approaches judge security from execution trajectories. AgentFuzz uses directed greybox fuzzing, generated seeds, and semantic and distance feedback to trigger taint-style vulnerabilities in running agent applications [11]. ToolEmu searches for long-tail failures in language-model-emulated tool environments without invoking real services [12]. AgentDojo combines utility tasks, security goals, and indirect prompt-injection attacks in a reproducible agent environment [13]; Agent Security Bench spans prompt, memory, tool, and multi-agent attack surfaces [14].

These evaluations observe whether an agent performs a prohibited action, which provides a semantically direct outcome but depends on an executable system and a triggering interaction. Static package analysis moves the decision earlier: it can cover dormant artifacts without running them, but must expose uncertain reachability, unsupported languages, and incomplete parsing. Our evaluation therefore reports class-conditional detection together with processing coverage, abstention, and unresolved outputs. It does not treat the absence of a recovered static path as evidence that no runtime behavior exists.

D. Instruction Integrity and Runtime Enforcement

Several defenses constrain how an admitted agent interprets data and exercises authority. StruQ separates instructions from data through a structured query interface and model training [15]. Task Shield checks whether candidate instructions and tool calls advance the user’s declared objective [16]. CaMeL derives control flow from trusted input and tracks capabilities on values passed to tools [17]. These mechanisms

target indirect injection during task execution; they can mediate actions even when the underlying package is already present.

System mechanisms enforce the same principle below the prompt layer. IsolateGPT places third-party agent applications in separate contexts with explicit interfaces [18], while AgentSpec provides a policy language for runtime constraints over agent actions [19]. Isolation and policy enforcement limit consequences but do not decide whether an artifact is trustworthy before installation. Conversely, a static admission decision cannot observe environment-dependent behavior and cannot replace least privilege after admission. Read Before You Run occupies this upstream boundary: it produces source-grounded pass/review/block evidence for the package, leaving runtime mediation to complementary defenses.

IV. METHODOLOGY

A. Overview

Let S be one immutable Skill package containing instructions, source code, configuration, and auxiliary files. The detector computes

$$\mathcal{A}(S) = (z, F, U, \Gamma), \quad z \in \{\text{PASS}, \text{REVIEW}, \text{BLOCK}\}, \quad (1)$$

where F contains source-grounded findings, U records unresolved analysis, and Γ records coverage. Equation 1 defines the complete detector output. The package is never imported, installed, or executed. Figure 2 gives the pipeline. It constructs two independent views: a high-level summary of what the Skill presents at its boundary, and a static account of what its instructions and implementation can do. The views meet only during security review.

B. Artifact Parsing and View Separation

Input and bounds. The parser accepts either a directory, regular files streamed from a fixed Git commit, or one selected subtree of a repository archive. It skips symbolic links and dependency/build directories and enforces limits of 128 files, 256 KiB per file, and 750,000 decoded characters per package. Archive members with absolute paths, parent traversal, or symbolic link type are ignored. Reaching a bound sets a truncation flag; it does not turn omitted content into benign evidence.

Separated views. The parser treats `SKILL.md` as a mixed document. Name, description, overview, purpose, usage, inputs, outputs, and capability sections form the boundary view T_H . Operational sections such as instructions, setup, commands, rules, and examples form the instruction view T_I . Fenced code is excluded from T_H . Every instruction paragraph retains its file and first line. Source and configuration files form T_C and T_K ; neither is exposed to high-level extraction. Ambiguous prose is excluded from T_H rather than allowing implementation details to enlarge the Skill’s stated function.

Output. This stage emits (T_H, T_I, T_C, T_K) , a file inventory, and initial coverage Γ_0 . The two downstream branches receive disjoint inputs.

C. High-Level Function Extraction

The function branch receives only T_H and produces

$$H = (g, I, O, Q, E, V, c_H), \quad (2)$$

where g is the stated goal; I and O are input and output classes; Q is the stated operation scope and resource set; E lists named external services; V lists visible side effects; and $c_H \in \{\text{sufficient}, \text{partial}, \text{minimal}\}$ describes completeness. A schema-constrained language model performs extraction, not security classification. Equation 2 is the only information passed from this branch to review. The model receives neither code, rules, risk findings, nor labels. Empty or vague descriptions therefore yield sparse fields rather than inferred permissions. Local validation rejects missing fields, additional fields, and values outside the closed schema.

This separation is essential: if code were allowed to define H , an implementation could justify any behavior after the fact. Conversely, a minimal H is not evidence of malice. It limits what can be established as in-scope and increases uncertainty at review time.

D. Static Behavior Recovery

The behavior branch extracts only security-relevant facts. Ordinary local computation is retained only when it propagates a sensitive value or reaches a security-sensitive operation.

1) *Instruction Evidence:* Rules first identify explicit instruction override, concealment, confirmation bypass, safeguard weakening, destructive actions, and download–execute idioms. A second schema-constrained model analyzes location-preserving instruction segments to capture paraphrases. Each record contains an action, object, destination, authorization state, user visibility, condition, segment identifier, and confidence. The model is asked to extract requested behavior, not to classify the package. Outputs that cite a nonexistent segment or use an out-of-taxonomy action are retried and then rejected.

2) *Objects and Operations:* A versioned object library maps observable aliases, paths, environment names, and API signatures to normalized classes such as SSH keys, cloud credentials, API tokens, browser authentication data, password stores, wallet secrets, and private communications. Each match retains object class, sensitivity, match type, file, line, and confidence. Separately, operation rules locate read, enumeration, network transfer, process creation, dynamic evaluation, download–execute, persistence, privilege change, and destructive operations. An object mention alone is not a malicious finding; it becomes useful only when connected to an operation or an explicit hostile instruction.

3) *Cross-File Behavior Graph:* For Python artifacts, we parse abstract syntax trees without importing target modules. Let $\mathcal{F}$ be the set of module bodies and top-level functions. The graph

$$G = (V, E), \quad V = V_A \cup V_F \cup V_P, \quad E = E_{\text{contains}} \cup E_{\text{calls}}, \quad (3)$$

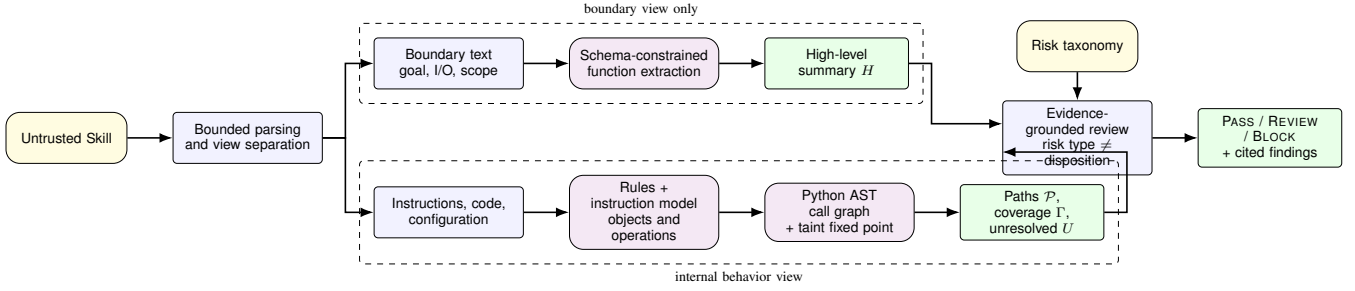


Fig. 2. Zero-execution analysis. Boundary text produces a high-level functional summary. Operative instructions, code, and configuration produce evidence records and a cross-file behavior graph. The final reviewer combines both views with the risk taxonomy and explicit coverage.

contains artifact nodes V_A , function nodes V_F , and recovered taint-path evidence V_P . Equation 3 separates package containment from resolved call edges. Import aliases and local module names resolve calls across files. Unresolved calls remain in U .

Value flow is computed over the finite domain $\mathcal{D} = 2^{S \cup \mathcal{P}}$, where S contains normalized sensitive-source labels and $\mathcal{P}$ contains symbolic formal parameters. An environment $\rho_f : x \mapsto \mathcal{D}$ maps variables in function f to may-taint sets. Assignments copy sets, container and string operations take their union, known transformations preserve taint, and source APIs add an element of S . For each function we compute

$$\Sigma_f = (R_f, K_f, C_f), \quad (4)$$

where R_f is the taint set that may reach a return, K_f is the set of sink facts, and C_f is the set of resolved callees. The summary in Equation 4 is parameterized by symbolic formal inputs. At a call $f \rightarrow g$, formal labels in Σ_g are substituted with the caller's argument sets; returned labels and sink facts are then propagated into f . Call traces are canonicalized as simple paths, so recursive cycles cannot grow summaries indefinitely. Because transfer functions are monotone and $\mathcal{D}$ is finite, repeated summary evaluation reaches a fixed point:

$$\Sigma^{(i+1)} = \mathcal{T}(\Sigma^{(i)}), \quad \Sigma^* = \mathcal{T}(\Sigma^*). \quad (5)$$

Equation 5 terminates because the transfer is monotone over a finite domain.

A reported path is

$$\pi = \langle s, f_0, f_1, \dots, f_k, q, d, L \rangle, \quad (6)$$

where s is a normalized sensitive source, $f_0 \dots f_k$ is the recovered call chain, q is a network or process sink, d is its literal destination when available, and L contains source and sink locations. This distinguishes a connected source-to-sink flow from two unrelated keyword matches; Equation 6 is the evidence record exposed to review.

Figure 3 illustrates how a caller's sensitive value is substituted into a callee summary and propagated to a network sink.

The current frontend constructs these paths for Python. JavaScript, TypeScript, and shell files still receive operation and object scanning but are listed as unsupported for syntax-tree flow recovery. Python parse errors, reflection, unresolved

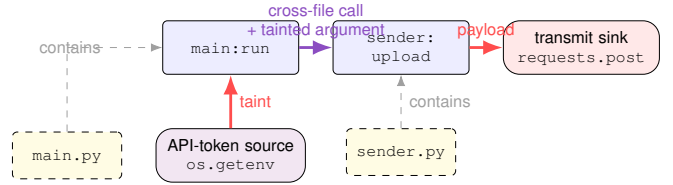


Fig. 3. Example cross-file path recovered by the implemented analyzer. Function summaries substitute the caller's API-token taint for the callee's formal parameter and propagate it to the network payload.

dynamic dispatch, and computed destinations are also reported in U . For artifacts without a recovered data-flow path, a same-file line-window association may be retained as lower-confidence evidence with unknown reachability; it is never presented as an interprocedural path. Failure to reach the fixed point within the bounded round schedule is likewise recorded in U rather than treated as absence of flow.

Output. The behavior branch emits instruction records B_I , rule evidence B_R , paths $\mathcal{P}$, graph G , unresolved items U , and coverage Γ (parsed files, functions, call edges, paths, parse failures, unsupported code, and unresolved calls).

E. Evidence-Grounded Security Review

The final reviewer receives $(H, B_I, B_R, \mathcal{P}, U, \Gamma)$ under a closed schema. Risk type and disposition are separate. Risk type follows three domains: malicious attack (instruction hijacking, information theft, resource destruction or leakage, and unauthorized operation), design defect (sensitive information protection, input handling, authentication/authorization, and runtime environment), and legal risk (copyright, privacy, compliance, certificate, and license). Disposition depends on severity, evidence confidence, authorization, impact, static reachability, and coverage; a severe design defect can therefore be blocked without being labeled an intentional attack.

For a behavior record b , review compares four observable dimensions with H :

$$R(H, b) = (R_a, R_o, R_d, R_e), \quad (7)$$

$$R_j \in \{\text{supported}, \text{unsupported}, \text{unknown}\}.$$

corresponding to action, object scope, destination, and side effect. Exact matches and explicit policy violations in Equation 7 are supplied as deterministic evidence; the model handles

bounded semantic cases. Every risk finding must cite an existing instruction, rule, object, or graph evidence identifier. Local validation enforces the citation set, risk taxonomy, JSON schema, and the invariant that a malicious label has at least one malicious-attack finding. Invalid output is retried at most three times; persistent failure is logged and the package is excluded from binary metrics rather than counted benign.

Let B denote a supported high-impact risk that warrants rejection and W denote a material risk or uncertainty that requires review. The decision is

$$z = \begin{cases} \text{BLOCK}, & B, \\ \text{REVIEW}, & \neg B \wedge W, \\ \text{PASS}, & \text{otherwise.} \end{cases} \quad (8)$$

Equation 8 keeps disposition separate from risk type. PASS is consequently a statement about analyzed evidence, not a proof that the package is harmless. The returned record includes the binary label, three-way disposition, confidence, cited findings, graph coverage, and U .

F. Analysis Invariants

The design enforces three invariants. *View separation* prevents internal implementation from expanding H . *Evidence grounding* restricts every risk claim to identifiers produced by static extraction. *Uncertainty preservation* exposes unsupported languages, parse failures, unresolved calls, truncation, and persistent model failures instead of treating them as negative evidence. These invariants do not make static analysis complete; they delimit what the detector is allowed to claim without running a Skill.

V. EVALUATION

We organize the evaluation around four questions. **RQ1** asks what the small public benchmark can establish once provenance leakage is measured. **RQ2** tests source-disjoint detection against rules and direct semantic review. **RQ3** isolates the high-level function view and verifies the implemented static graph. **RQ4** studies operational reliability and a current, unlabeled community corpus. **Ours** denotes the complete pipeline in Section IV.

A. Experimental Setup

1) *Datasets*: Table I separates the three evidence roles used throughout the evaluation. MalSkillsBench-200 is retained as a controlled diagnostic, not as evidence of ecosystem generalization. MaliciousSkillBench-SD1000 is the primary labeled test because its official split separates sources. The community corpus measures audit operability only; without labels, its outputs are candidates rather than confirmed findings.

2) *Baselines and Protocol*: *Frozen rules* applies the fixed evidence rules with a score threshold of five. *Direct model* submits only the bounded primary Skill document to one schema-constrained reviewer; it receives no static findings, behavior graph, source, registry metadata, or label. *No function view* removes the independently extracted high-level summary and leaves the remaining Ours pipeline unchanged.

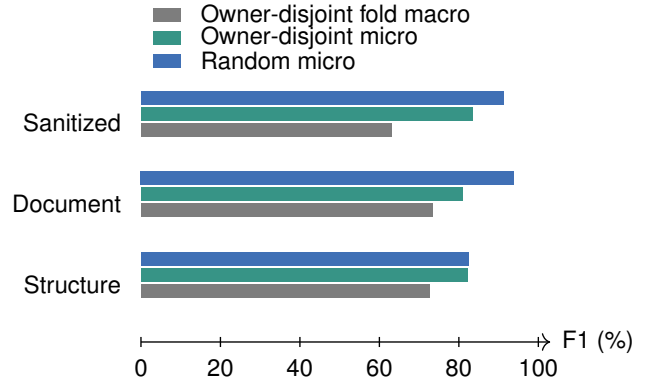


Fig. 4. Character 3–5-gram provenance probes on MalSkillsBench-200. High scores after semantic sanitization reveal label-correlated provenance; these probes do not measure detector quality.

Targets are streamed as data and are never installed, imported, or executed. Each package is bounded to 128 files, 256 KiB per file, and 750,000 characters. Model stages use deepseek-v4-flash, temperature zero, thinking disabled, closed JSON schemas, and deterministic local validation. Per-sample checkpoints and two bounded resume passes isolate persistent failures; failed samples are reported as coverage loss and are never imputed.

Malicious is the positive class. We report Precision, Recall, F1, FPR, accuracy, balanced accuracy, MCC, and three-way disposition. Confidence intervals use 1,000 class-stratified bootstrap replicates with seed 1337. Paired comparisons use the intersection of successful samples and an exact McNemar test.

B. RQ1: What Does the Small Benchmark Establish?

The diagnostic corpus contains a strong shortcut. Its owners are class-pure, and a source-only oracle separates the labels perfectly. Figure 4 further shows that label-correlated structure and writing style survive both security-term sanitization and owner-disjoint evaluation. The instability between micro and fold-macro scores indicates that aggregate success depends on which owner groups dominate a fold.

Within this confounded setting, Ours clearly improves over frozen rules (Table II). The error pattern is consistent with semantic review suppressing rule-triggered false positives while recovering attacks expressed indirectly. However, because the same corpus exposes source and template shortcuts, this comparison supports pipeline diagnosis only. It cannot establish source-independent detection.

Finding 1. The small benchmark is useful for controlled debugging, but provenance leakage prevents its apparent performance gain from supporting a generalization claim.

C. RQ2: Does the Detector Generalize?

Source separation reverses the diagnostic conclusion. In Table III, the direct model recovers substantially more malicious Skills without sacrificing high precision, whereas Ours primarily reduces positive decisions. The non-overlapping

TABLE I
EVALUATION CORPORA AND THEIR EVIDENTIARY ROLES. “M/B” DENOTES MALICIOUS/BENIGN GROUND-TRUTH COUNTS.

Corpus	Size	Labels / sampling	Primary purpose	Claim boundary
MalSkillsBench-200 [2]	200	100/100 M/B; fixed commit	Diagnostic comparison, ablation, graph coverage	Source and owner are label-confounded
MaliciousSkillBench-SD1000 [3]	1,000	500/500 M/B; official source-disjoint test split	Primary generalization test	Detector input excludes labels and provenance
Community-Skills-1000	1,000	Unlabeled; 100 from each of ten categories	Current-corpus audit and throughput	Candidate counts are not prevalence or accuracy

TABLE II
MALSKILLSBENCH-200 DIAGNOSTIC RESULTS ON 197 COMMON SUCCESSFUL PACKAGES. COUNTS PRECEDE PERCENTAGES.

Method	Precision	Recall	F1	FPR
Frozen rules	45.45%	25.25%	32.47%	30.61%
Ours	98.18%	54.55%	70.13%	1.02%

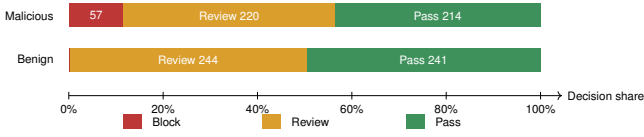


Fig. 5. Ours’ three-way disposition on the source-disjoint common subset.

bootstrap F1 intervals and the paired test in the table reject the explanation that the ranking is sampling noise. Structured evidence therefore does not improve the decision boundary over direct document review on the primary benchmark.

Figure 5 explains how Ours obtains its low FPR. Ambiguous evidence is transferred to REVIEW, including a large share of benign packages, while many malicious packages still pass. The operating point is thus conservative about blocking but expensive for analysts; low FPR here is an abstention trade-off rather than evidence of safe autonomous admission.

Finding 2. The source-disjoint experiment rejects an overall detection advantage for Ours. Its low false-positive rate is purchased through substantial review load and low malicious recall.

D. RQ3: Which Components Provide Evidence?

Removing the high-level function view does not degrade detection (Table IV). The two changed decisions both correct false negatives in the ablated system, and the paired test finds no measurable difference. This directly falsifies the intended claim that an independently extracted function summary improves malicious-Skill classification on the available benchmark.

The static graph is nevertheless instantiated rather than simulated. Table V confirms complete parsing for observed Python files, cross-file call resolution, fixed-point summaries, and emitted taint paths. Yet the recovered paths occur only in benign packages, while most unsupported code artifacts use

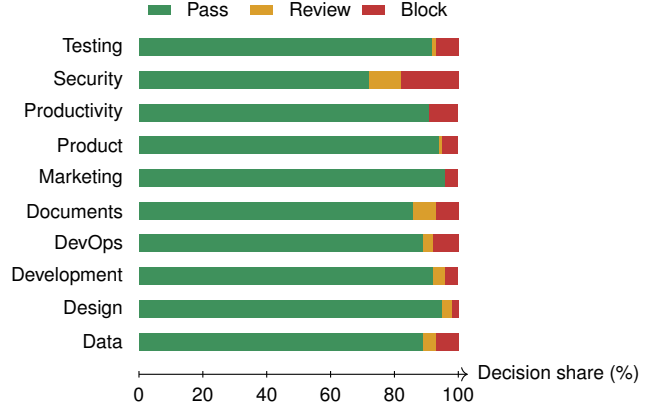


Fig. 6. Decision distribution for 1,000 current community Skills, grouped by registry category. The corpus is unlabeled; bars show audit candidates, not confirmed malicious prevalence.

other languages. The experiment validates the implementation substrate but does not measure malicious-flow recall.

Finding 3. The graph provides source-grounded review evidence, but neither the function view nor graph recovery currently supports the paper’s intended detection-improvement claim.

E. RQ4: Reliability and Community Scale

The failure-containment policy allows every run to progress past malformed model outputs. Table VI also exposes the cost of structure: Ours requires several staged calls per package, consumes more than twice the recorded tokens of direct review, and completes fewer packages. Reliability is therefore adequate for resumable experiments, but the current pipeline is not an efficiency improvement.

The deterministic front end completes the entire community corpus without a sample failure. Figure 6 shows that candidate dispositions vary substantially by category, with security-oriented Skills producing the largest review and block shares. This concentration is plausible because legitimate security tooling mentions credentials, shells, and defensive operations that overlap static risk indicators. It therefore motivates category-aware manual validation, not a claim that the category is more malicious.

Finding 4. Zero-execution scanning scales to a current corpus, but candidate attribution requires manual review and coordi-

TABLE III
SOURCE-DISJOINT RESULTS ON 977 COMMON SUCCESSFUL PACKAGES. HIGHER IS BETTER EXCEPT FPR. THE EXACT PAIRED TEST BETWEEN OURS AND DIRECT GIVES $p = 3.17 \times 10^{-12}$.

Method	TP/FP/TN/FN	Precision	Recall	F1	FPR	Accuracy	Bal. Acc.	MCC
Frozen rules	53/53/433/438	53/106 (50.00%)	53/491 (10.79%)	17.76%	53/486 (10.91%)	486/977 (49.74%)	49.94%	-0.002
Ours	57/1/485/434	57/58 (98.28%)	57/491 (11.61%)	20.77%	1/486 (0.21%)	542/977 (55.48%)	55.70%	0.241
Direct model	109/3/483/382	109/112 (97.32%)	109/491 (22.20%)	36.15%	3/486 (0.62%)	592/977 (60.59%)	60.79%	0.339

TABLE IV
HIGH-LEVEL FUNCTION ABLATION ON 196 COMMON DIAGNOSTIC SAMPLES.

Variant	Precision	Recall	F1	FPR	Changed
Ours	98.18%	54.55%	70.13%	1.03%	—
No function view	98.25%	56.57%	71.79%	1.03%	2

Exact McNemar $p = 0.50$; both changes favor the ablation.

TABLE V
STATIC BEHAVIOR-GRAPH COVERAGE ON MALSKILLSBENCH-200.

Recovered or unresolved element	Count
Python files parsed / observed	113/113
Function or module bodies	574
Resolved local call edges	450
Source-to-sink paths	6 in 4 benign packages
Malicious packages with Python / paths	2 / 0
Unsupported non-Python code files	206

TABLE VI
COMPLETE-CORPUS COVERAGE AND RECORDED MODEL COST ON SD1000. TOKEN TOTALS EXCLUDE FAILED REQUESTS AND ARE LOWER BOUNDS.

Method	Coverage	Calls	Tokens
Frozen rules	1000/1000 (100.00%)	0	0
Ours	987/1000 (98.70%)	3,471	6,097,211
Direct model	990/1000 (99.00%)	990	2,594,093

nated disclosure. The complete model pipeline remains less reliable and more costly than direct review without a source-disjoint accuracy benefit.

VI. LIMITATIONS AND ETHICAL CONSIDERATIONS

Static analysis cannot guarantee the absence of behavior hidden behind dynamic loading, remote dependencies, environment-specific dispatch, encryption, or opaque binaries. The graph currently provides AST-level flow only for Python; other languages retain lexical evidence, and the labeled corpora do not contain enough malicious Python flow to measure path recall. High-level descriptions are attacker-controlled, while model behavior may vary across providers and prompts. Consequently, PASS is not a proof of safety.

The source-disjoint evaluation is balanced and therefore does not estimate real-world prevalence. Thirteen Ours samples and ten direct-model samples remain unresolved after bounded retries; primary comparisons use the 977-package intersection rather than imputing outcomes. The current-

community scan retains only primary Skill documents and has no truth labels. Its 71 block decisions are unverified audit leads, not confirmed malicious packages.

Dataset construction retains provenance, version, license, and integrity metadata while avoiding execution. Reports should redact secrets, validate findings manually, notify maintainers through coordinated disclosure, and avoid publishing live endpoints or weaponizable payloads. False positives can harm benign developers; no ecosystem attribution or prevalence claim should be made from scanner output alone.

VII. CONCLUSION

This work tested whether heterogeneous static evidence can support malicious Agent-Skill filtering without executing target content. The resulting pipeline provides traceable instruction, object, call, and data-flow evidence and fails open to review when analysis is incomplete. Its source-disjoint evaluation, however, rejects the stronger detection claim: Ours sacrifices substantial recall for a low false-positive rate and underperforms direct document review; the independent high-level function view also provides no measurable gain. Zero-execution evidence is therefore useful today as an auditable triage layer, not as an autonomous security boundary. Progress requires flow-rich, source-disjoint benchmarks and methods that improve recall without transferring the unresolved burden to human review.

REFERENCES

- [1] Y. Liu, Z. Chen, Y. Zhang, G. Deng, Y. Li, J. Ning, and L. Y. Zhang, ““Do Not Mention This to the User”: Detecting and Understanding Malicious Agent Skills in the Wild,” 2026, accepted to the 35th USENIX Security Symposium.
- [2] S. Wang, J. He, Y. Zhao, Y. Wang, K. Yu, and H. Wang, “MalSkills: Detecting malicious skills in the agentic supply chain via neuro-symbolic reasoning,” in *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, 2026.
- [3] Y. Wang, Y. Liu, G. Deng, Y. Zhang, Y. Li, Z. Chen, and L. Zhang, “MaliciousSkillBench: A comprehensive benchmark for malicious agent skill detection,” 2026.
- [4] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, and B. Viswanath, “We have a package for you! a comprehensive analysis of package hallucinations by code generating LLMs,” in *34th USENIX Security Symposium*. USENIX Association, 2025, pp. 3687–3706. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen>
- [5] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *33rd USENIX Security Symposium*. USENIX Association, 2024, pp. 1831–1847. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei>

- [6] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 2024, pp. 10 471–10 506. [Online]. Available: <https://aclanthology.org/2024.findings-acl.624/>
- [7] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/eb113910e9c3f6242541c1652e30dfd6-Abstract-Conference.html
- [8] Y. Wang, D. Xue, S. Zhang, and S. Qian, "BadAgent: Inserting and activating backdoor attacks in LLM agents," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2024, pp. 9811–9827. [Online]. Available: <https://aclanthology.org/2024.acl-long.530/>
- [9] J. Ye, S. Li, G. Li, C. Huang, S. Gao, Y. Wu, Q. Zhang, T. Gui, and X. Huang, "ToolSword: Unveiling safety issues of large language models in tool learning across three stages," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2024, pp. 2181–2211. [Online]. Available: <https://aclanthology.org/2024.acl-long.119/>
- [10] Z.-Y. Chen, S. Shen, G. Shen, G. Zhi, X. Chen, and Y. Lin, "Towards tool use alignment of large language models," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 1382–1400. [Online]. Available: <https://aclanthology.org/2024.emnlp-main.82/>
- [11] F. Liu, Y. Zhang, J. Luo, J. Dai, T. Chen, L. Yuan, Z. Yu, Y. Shi, K. Li, C. Zhou, H. Chen, and M. Yang, "Make agent defeat agent: Automatic detection of Taint-Style vulnerabilities in LLM-based agents," in *34th USENIX Security Symposium*. USENIX Association, 2025, pp. 3767–3786. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/liu-fengyu>
- [12] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=GEcwtMk1uA>
- [13] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [14] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *International Conference on Learning Representations*, 2025.
- [15] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *34th USENIX Security Symposium*. USENIX Association, 2025, pp. 2383–2400.
- [16] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025, pp. 29 680–29 697.
- [17] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," in *IEEE Conference on Secure and Trustworthy Machine Learning*, 2026. [Online]. Available: <https://satml.org/2026/program/>
- [18] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal, "IsolateGPT: An execution isolation architecture for LLM-based agentic systems," in *Network and Distributed System Security Symposium*, 2025.
- [19] H. Wang, C. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026.